Guía de Calidad y Testing

Existe un mito peligroso: *"Si la IA escribe el código y le pido pruebas unitarias y todo pasa en verde, no necesito leer el código".*

Esto es un error garrafal. La calidad del código y la lectura exhaustiva son la única manera de prevenir "errores fantasma" (bugs silenciosos).

1. La IA No es una Muleta, es un Asistente

El objetivo de leer el código que la IA genera es que desarrolles un **Ojo Crítico**. Cuando prestas atención a los errores comunes, aprendes de ellos y, en tu siguiente instrucción, previenes que sucedan. Un ingeniero maduro pasa menos tiempo arreglando bugs porque, desde el Prompt inicial, le cerró las puertas al error.

2. Pruebas Unitarias Asistidas (TDD)

El *Test-Driven Development* (Desarrollo Guiado por Pruebas) brilla con la IA si lo usas en el orden correcto.

El Flujo Correcto:

1. **Define el Contrato:** Creas la interfaz.
2. **La IA escribe el Test:** Pides una prueba unitaria basada en ese contrato (ANTES de escribir la lógica) usando asserts lógicos.
3. **Revisión Humana:** Lees la prueba. ¿Contempla Casos Borde?
4. **La IA escribe la Lógica:** Pides código para que esa prueba pase en verde.
5. **Auditoría Final:** Lees el código generado para verificar modularidad.

Ejemplo Práctico: TDD con `calculateTotal`

Paso 1: Define el Contrato (Tú escribes esto)

```
interface CartItem { price: number; quantity: number; }
function calculateTotal(items: CartItem[], discountThreshold: number): number;
```

Paso 2: Pide el Test (ANTES de la lógica)

> *Prompt: "Escribe ÚNICAMENTE aserciones lógicas para calculateTotal validando: (1) total sin descuento bajo el umbral, (2) 10% de descuento sobre el umbral. No escribas la función, solo los tests."*

```
import { calculateTotal } from './cart';
import { assert } from './testing_utils';

// Caso 1: Total sin descuento
const itemsNormales: CartItem[] = [{ price: 20, quantity: 2 }];
assert(calculateTotal(itemsNormales, 100) === 40, "Debería ser 40");

// Caso 2: 10% de descuento
const itemsDescuento: CartItem[] = [{ price: 60, quantity: 2 }];
assert(calculateTotal(itemsDescuento, 100) === 108, "Debería ser 108");
```

Paso 3: Tú lees el test. ¿Faltan casos borde? (ej. carrito vacío).

Paso 4: La IA escribe la lógica. *"Escribe la función basándote en la interfaz para que el test pase en verde."*

Paso 5: Lees el código resultante. Aunque pase en verde, validas que sea pura.

3. Cero Deuda Técnica por Pereza

Es inadmisible que aceptes código desordenado cuando una IA puede generar *refactors* en segundos. Tu deber es identificar el problema y ordenarle: *"Funciona, pero está acoplado. Refactoriza extrayendo la lógica a una función pura independiente."*

> **Antes (Código IA, espagueti):**

```
function procesarPedido(userId: string, items: any[]) {
    let total = 0;
    for (const item of items) total += item.price * item.quantity;
    if (total > 100) total *= 0.9;
    await db.query(`INSERT INTO orders VALUES ('${userId}', ${total})`);
    return { ok: true, total };
}
```

> **Después (Tú exigiste refactorización SRP):**

```
// 1. Función pura de negocio
function calculateTotal(items: CartItem[], threshold: number) {
    const sub = items.reduce((s, i) => s + i.price * i.quantity, 0);
    return sub > threshold ? sub * 0.9 : sub;
}

// 2. Persistencia
async function saveOrder(userId: string, total: number) {
    await db.query('INSERT INTO orders...', [userId, total]);
}

// 3. Orquestador
async function procesarPedido(userId: string, items: CartItem[]) {
    const total = calculateTotal(items, 100);
    await saveOrder(userId, total);
    return { ok: true, total };
}
```

La diferencia no es estética: los archivos refactorizados son auditables y testeables. Si cultivas esto desde el día uno, el desarrollo será ágil y seguro.